

Programação Avançada

Unidade 2: Estruturas de Dados

Aula 12: Árvores Binárias de Busca

Prof. Eduardo Corrêa Gonçalves

15/06/2026

Sumário

Introdução

O que são Árvores e Árvores Binárias?

Terminologia

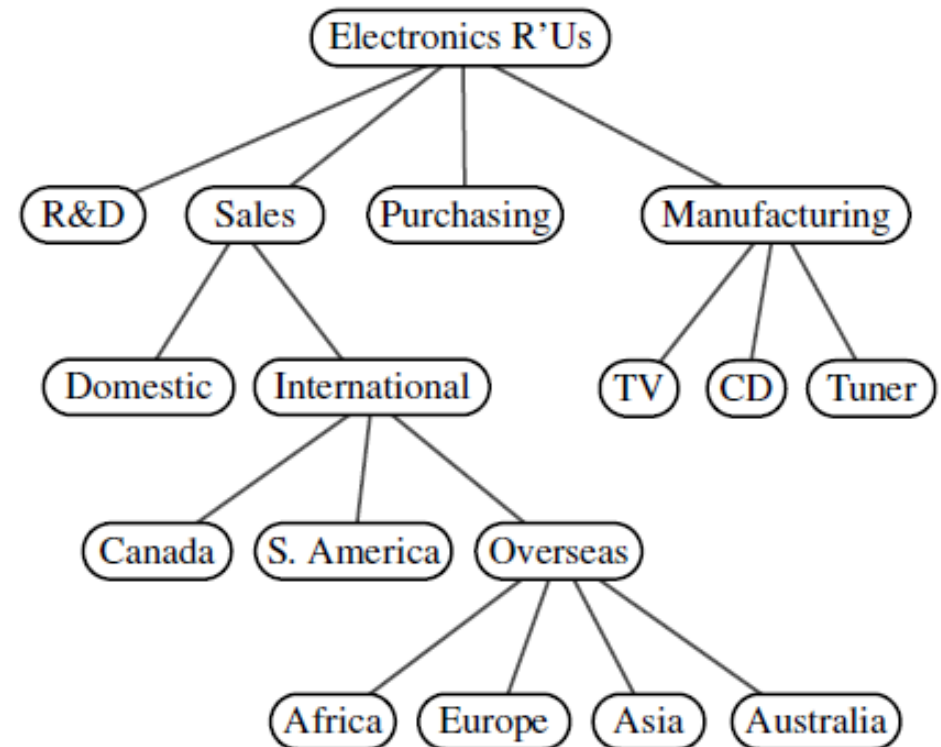
O que são Árvores Binárias de Busca (ABBs)?

Criação de ABBs

Operações com ABBs

Introdução (1/7)

- O que é Árvore?
 - Trata-se de uma forma de organizar dados.
 - É similar à lista encadeada, onde cada nó tem um link para o próximo.
 - Porém na árvore, cada nó pode ter **dois ou mais** links para outros nós.
 - Sendo assim, a árvore é uma **estrutura hierárquica**, ao contrário da lista que é linear.

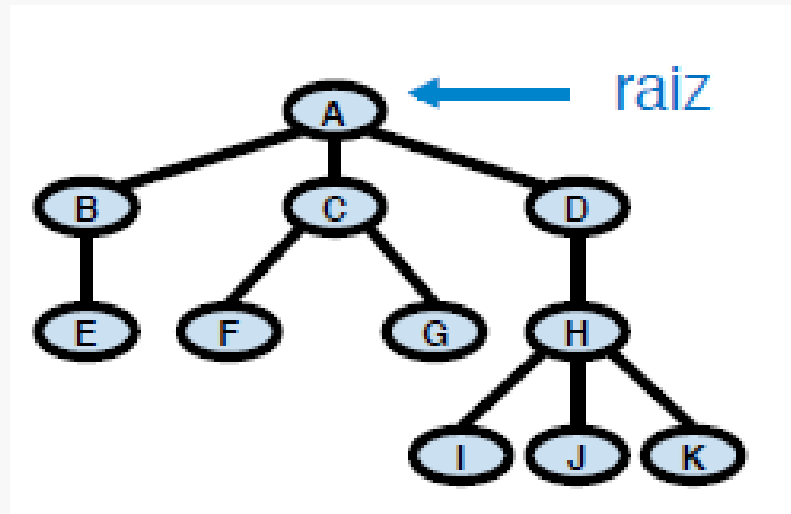


Exemplo: árvore com 17 nós representando a organização de uma empresa hipotética (Goodrich et al., 2013)

Introdução (2/7)

- Árvore - Definição Formal

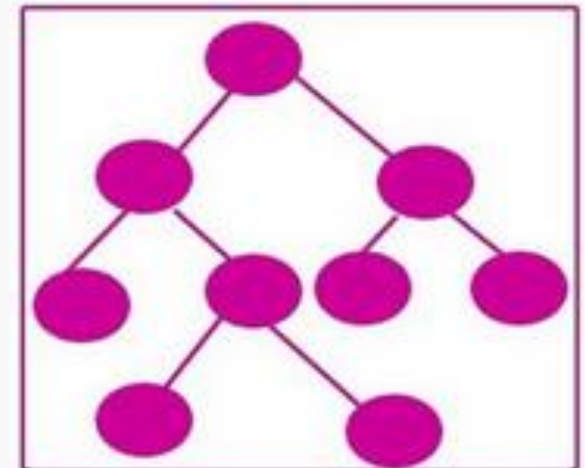
- Conjunto finito **T** de zero ou mais nós, tal que:
 - Se o número de nós é zero, trata-se de uma **árvore vazia**.
 - Senão
 - Existe um nó chamado **raiz**, denotado por $r(T)$
 - Os demais nós formam $m \geq 0$ conjuntos distintos $S_1, S_2, \dots, S_m$ onde cada um desses é uma árvore (S_i são denominadas **subárvores**).



Introdução (3/7)

- **Árvore – Características**

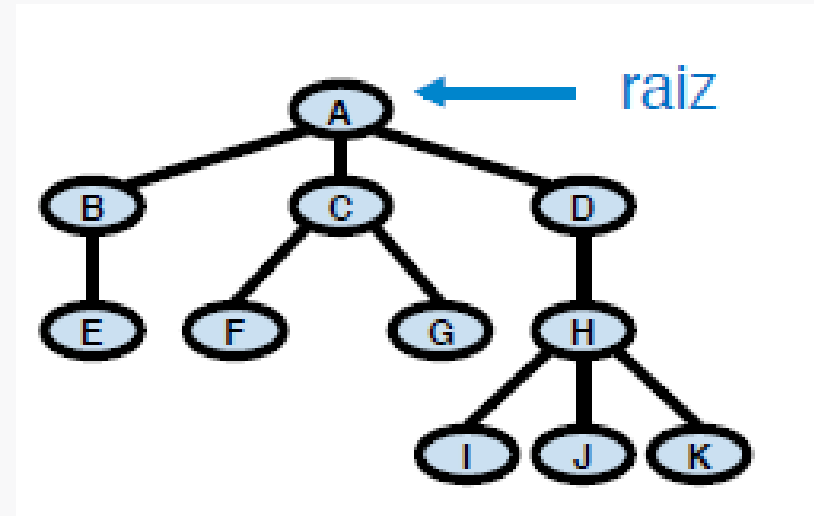
- Pode ou não permitir elementos **duplicados** (depende da aplicação)
- **Mutável** – pode ser alterada.
- **Não é Sequência** – pois os elementos não podem ser acessados através um índice que indica a sua posição ou ordem.
- **Iterável** – capaz de retornar seus elementos um por vez em um laço. Podemos iterar (nesse caso, “caminhar na árvore”) de várias formas, como será visto.
- **É um tipo de grafo** – a árvore é um **grafo conexo** (conecta todos os nós) e **acíclico** (não tem caminhos fechados)



Introdução (4/7)

- Terminologia

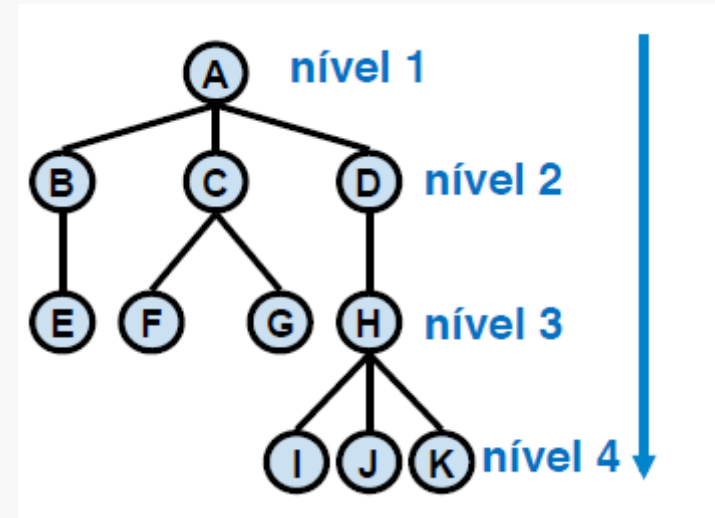
- A é o nó **raiz** (está no topo da árvore)
- B, C, D e H são **nós internos**.
- E, F, G, I, J, K são **nós folha**.



- O nó A tem 3 **sub-árvores**. Da mesma forma, B tem 1 subárvore, C tem 2 subárvores ,etc.
- A é **pai** de B, C e D. Da mesma forma B é pai de E, etc.
- B, C e D são **irmãos** (filhos de um mesmo pai), assim como F e G, etc.
- I, J, K são **filhos** de H, assim como B, C e D são filhos de A, etc.

Introdução (5/7)

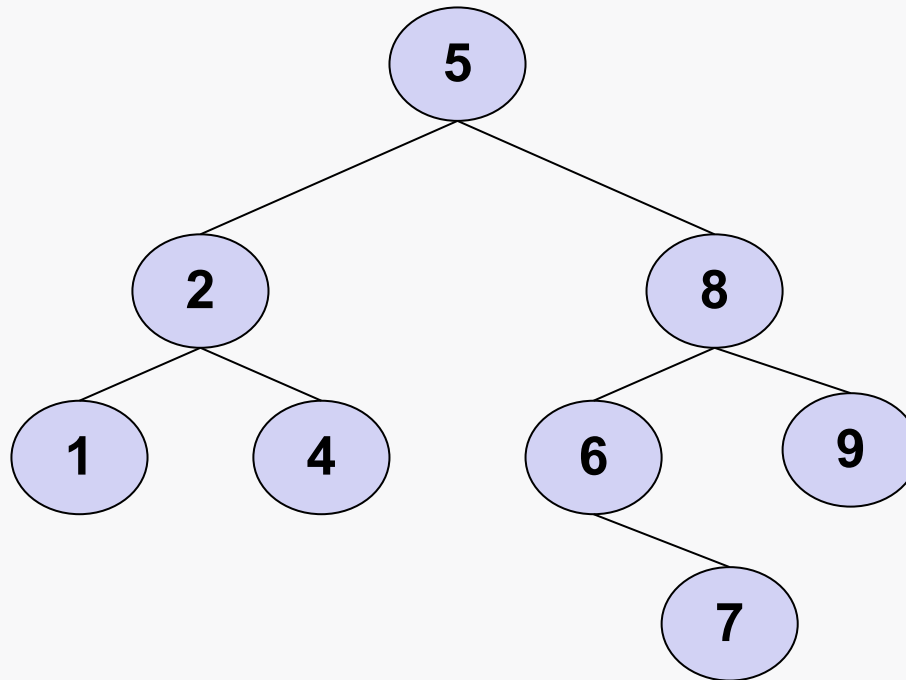
- **Terminologia**
 - **Nível:** número de ligações entre a raiz e o nó, acrescido de 1 unidade.
 - **Altura da árvore:** maior nível dentre seus nós
 - No exemplo ao lado, a altura da árvore é 4.



Introdução (6/7)

- **Aplicações**

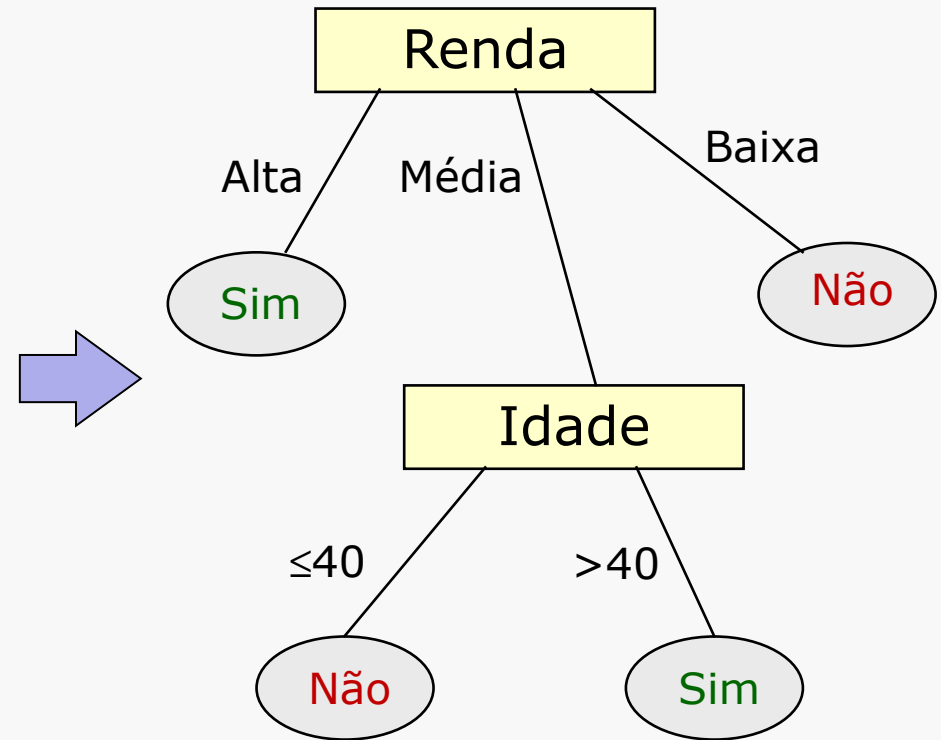
- As árvores são uma das estruturas mais importantes da ciência da computação devido ao seu grande número de **aplicações práticas**.
- **Exemplo 1**: são utilizadas para guardar dados ordenados.



Introdução (7/7)

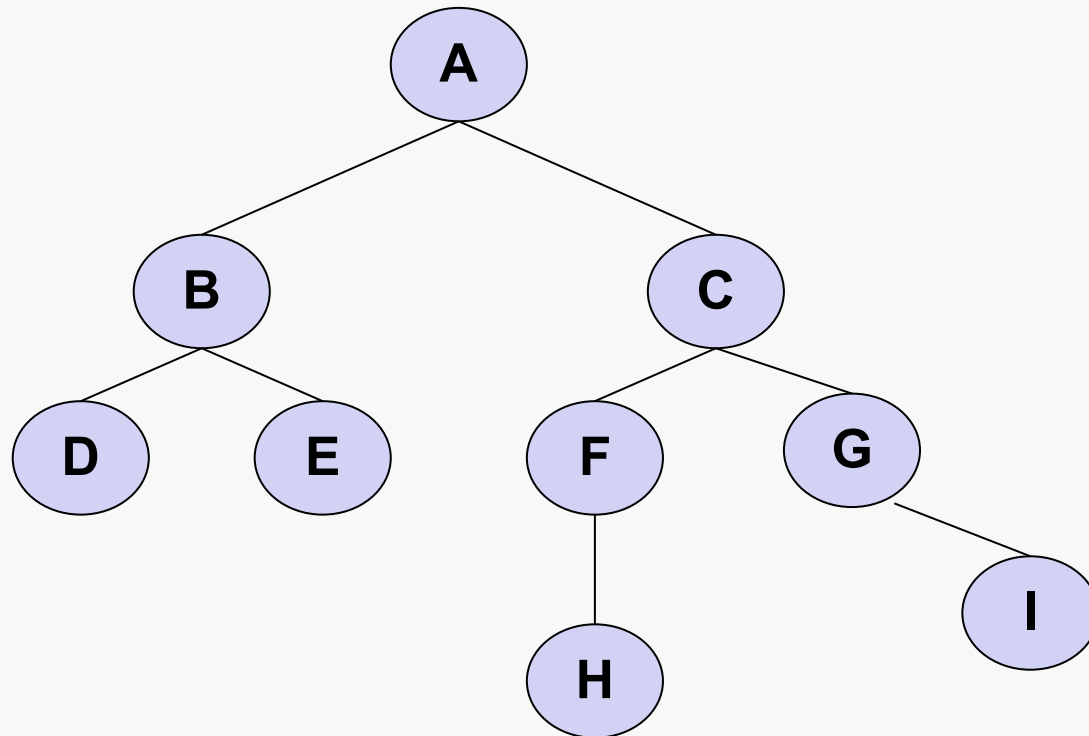
- Aplicações (Exemplo 2): Ciência de Dados
 - Na ciência de dados, os modelos de classificação e regressão baseados em **árvores de decisão** e **floresta aleatória** são muito populares.

Idade	Renda	Aluga Carro Importado?
38	Média	Não
41	Média	Sim
45	Alta	Sim
38	Baixa	Não
29	Média	Não
55	Média	Sim
18	Baixa	Não
50	Alta	Sim



Árvores Binárias (1/3)

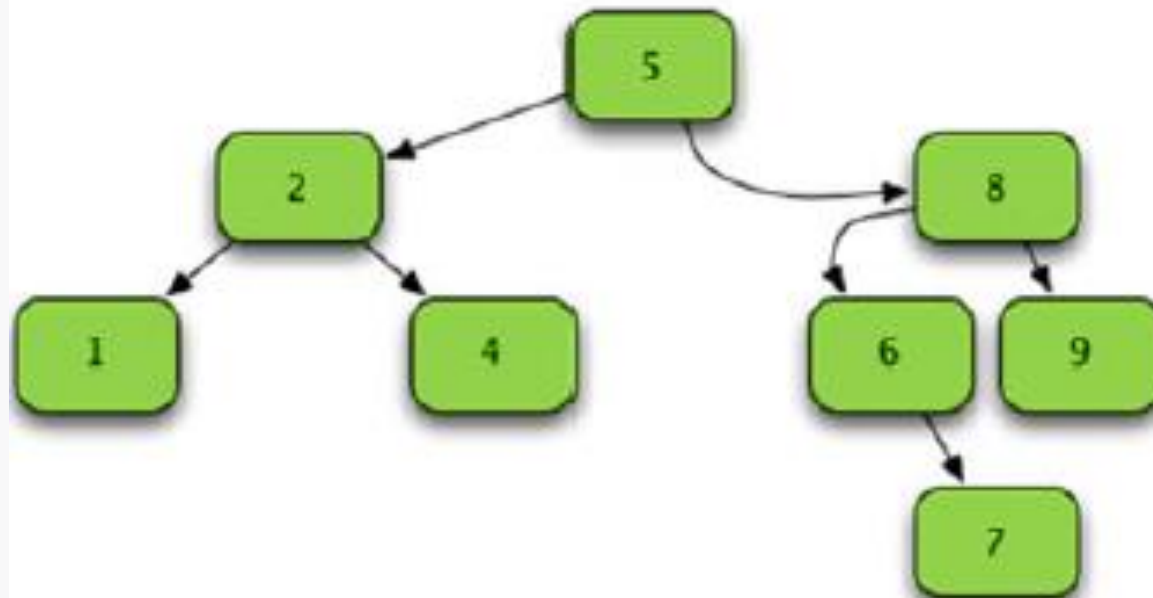
- O que é Árvore Binária?
 - É o tipo de árvore em que um nó pode ter **no máximo 2 links** para outros nós.



- Por outro lado, árvores onde os nós podem ter **mais de 2 links** para outros nós são chamadas de **n-árias**.

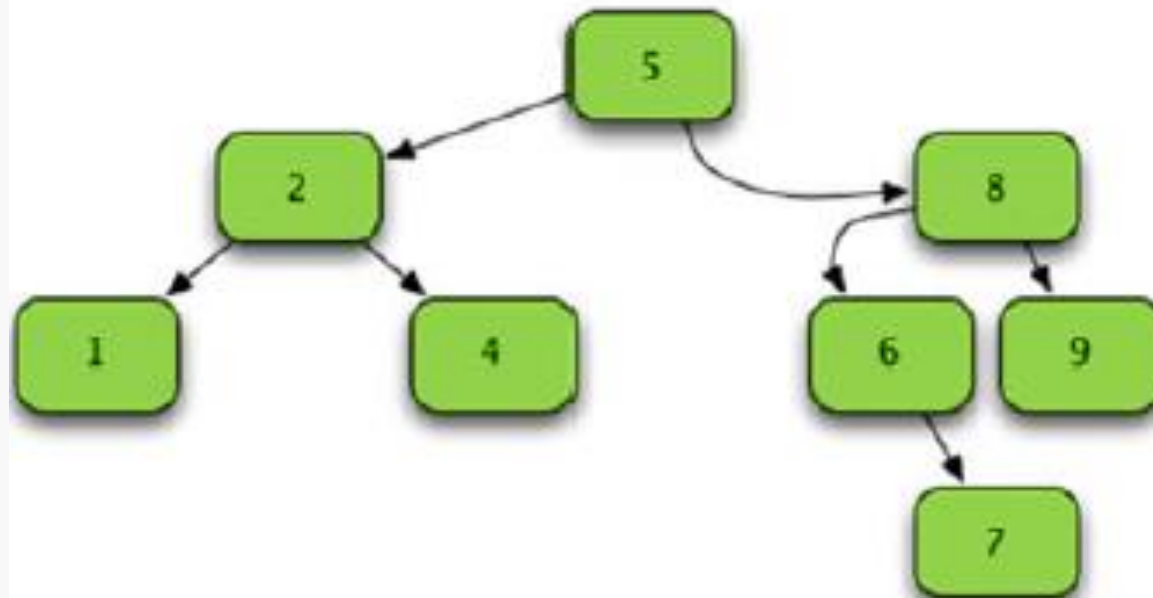
Árvores Binárias (2/3)

- **Árvore Binária de Busca (ABB)**
 - Nesta aula abordaremos um tipo especial de árvore binária denominada **ABB - árvore binária de busca** (*binary search tree - bst*), também chamada de Árvore Binária de Pesquisa (ABP)
 - Estas árvores possibilitam **inserir** valores e **buscar** por um determinado valor de forma eficiente.



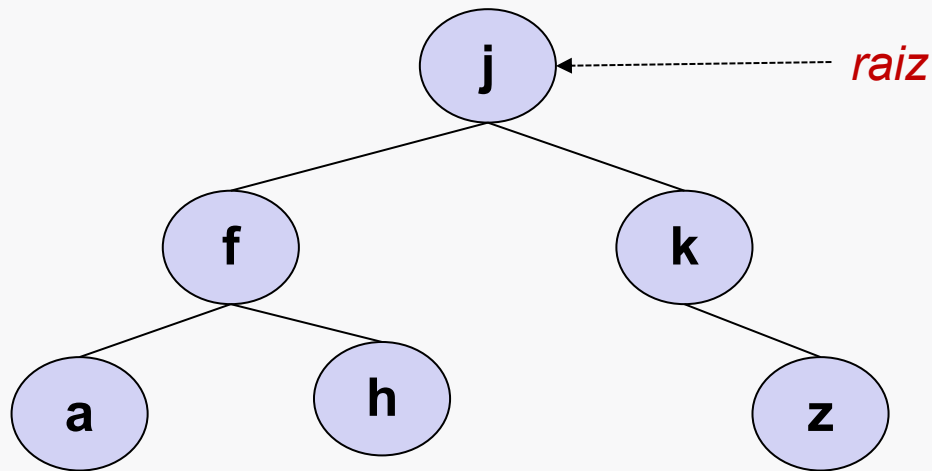
Árvores Binárias (3/3)

- **Árvore Binária de Busca (ABB)**
 - Cada nó tem no máximo 2 filhos (é uma árvore binária)
 - Todos os nós da subárvore à esquerda possuem valor menor que a raiz.
 - Todos os nós da subárvore à direita possuem valor maior que a raiz.
 - As subárvores à esquerda e direita também são ABBs



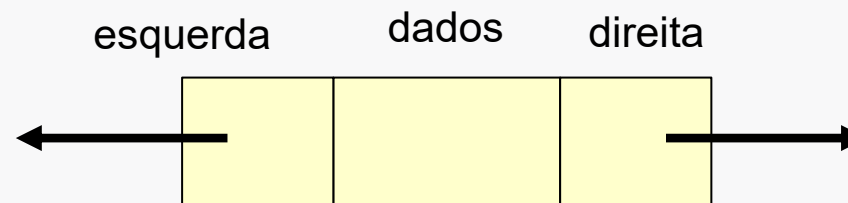
Criação de ABBs (1/3)

- Implementando a nossa ABB
 - Nesta aula, implementaremos a nossa própria ABB utilizando POO.



Criação de ABBs (2/3)

- Estrutura de um Nó da ABB
 - Em uma ABB, cada nó possui três diferentes campos:
 - **dados** (*data*): contém o valor do nó.
 - **esquerda** (*left*): contém a referência para o nó filho da esquerda.
 - **direita** (*right*): contém a referência para o nó filho da direita.



Criação de ABBs (3/3)

- É preciso definir duas classes:

- **No**: representação de um nó da árvore (dados + esquerda + direita)

class No:

```
def __init__(self, dados):
    self.info = dados
    self.esq = None
    self.dir = None
```

- **ABB**: p/ representar a árvore propriamente dita (cadeia de nós)

class ABB:

```
def __init__(self):
    #ponteiro para a raiz
    self.raiz = None

    #num. de elementos
    self.N = 0
```

Operações

- A ABB será implementada com os métodos apresentados na tabela abaixo.

Operação	Complexidade (caso médio)
Criação	$O(1)$
Inserir Nó	$O(\log n)$
Remover Nó	$O(\log n)$
Pesquisar se um valor existe	$O(\log n)$
Caminhamento (visitar todos os nós)	$O(n)$

- Todas as implementações serão **recursivas**
 - Embora seja possível implementar versões iterativas (não-recursivas) dos métodos, as implementações recursivas são bem mais comuns e simples.
- OBS.:** a busca em um vetor ordenado (busca binária) é $O(\log n)$. Porém, inserir valores em um vetor ordenado é $O(n)$. Daí a vantagem da ABB.

Operações – Inserir nó (1/10)

- Algoritmo – inserir_no(x) [insere novo nó com o valor x]
 - se a árvore estiver vazia:
 - inserir novo nó na raiz
 - se a árvore não estiver vazia:
 - compara x com o valor da raiz:
 - se x for menor, chama inserir_no(x) na subárvore da esquerda
 - caso contrário, chama inserir_no(x) na subárvore da direita
 - Inserção será feita quando **folha** for atingida.
- *** **IMPORTANTE:** na ABB, a inserção é sempre feita nas folhas

Operações – Inserir nó (2/10)

- Inserção – exemplo passo-a-passo

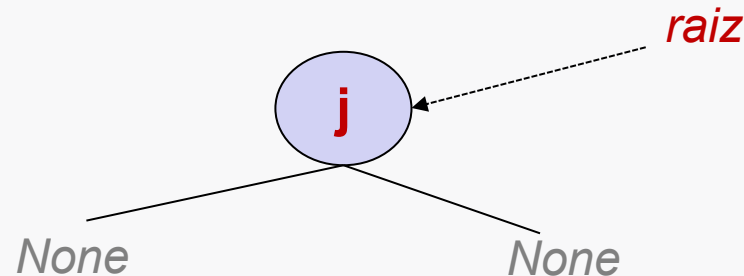
- Ao ser criada, a árvore está vazia:

raiz → *None*

- Vamos inserir a seguinte sequência de elementos:

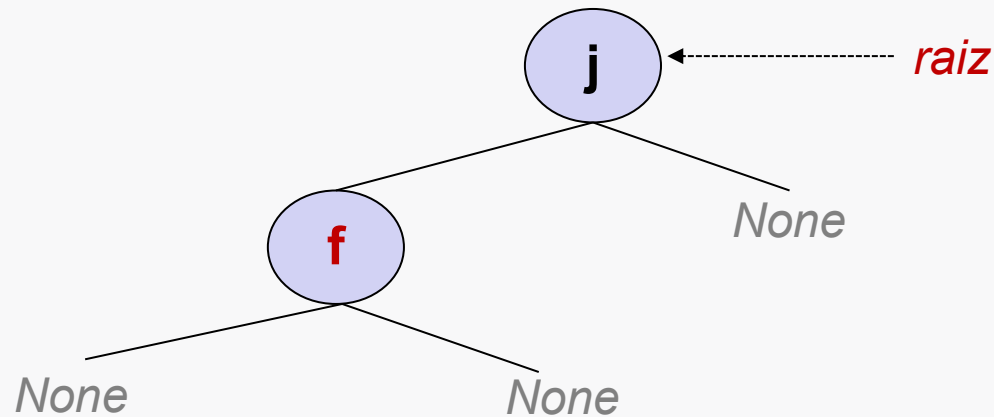
- **"j" – "f" – "k" – "a" – "z" – "h"**

- Ao inserir o primeiro nó e ele se torna a raiz: **inserir("j")**



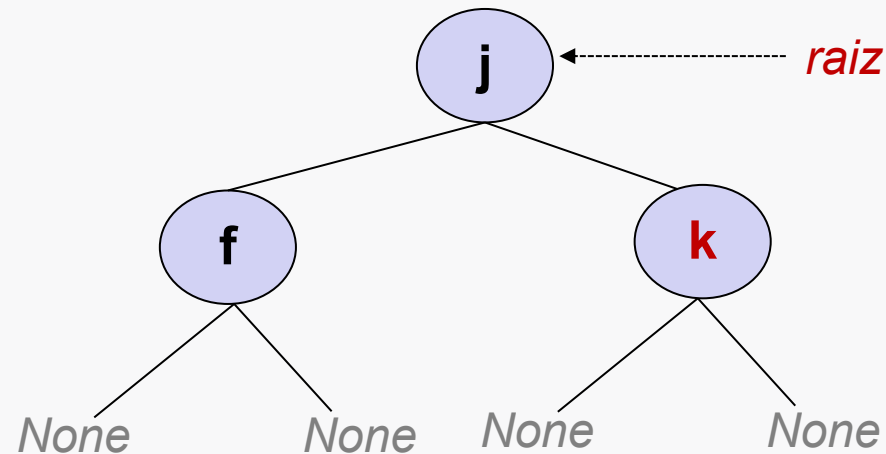
Operações – Inserir nó (3/10)

- Inserção – sequência "j" – "f" – "k" – "a" – "z" – "h"
- Inserindo o nó "f"
- Como "f" < "j", o nó é inserido à esquerda da raiz



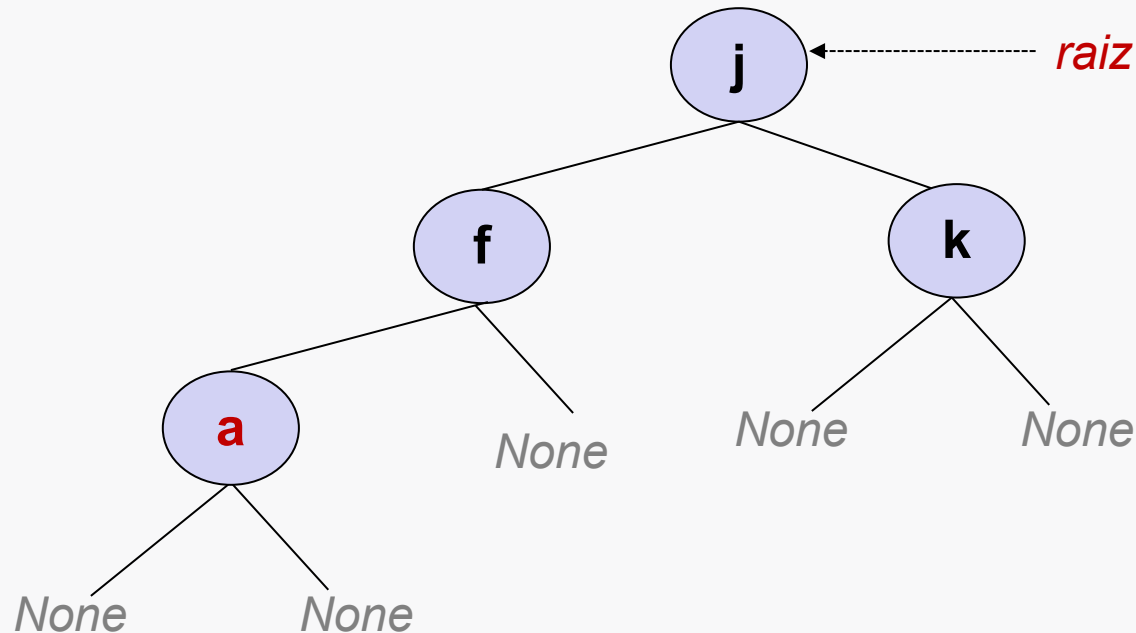
Operações – Inserir nó (4/10)

- Inserção – sequência "j" – "f" – "k" – "a" – "z" – "h"
 - Inserindo o nó "k"
 - Como "k" > "j", o nó é inserido à direita da raiz



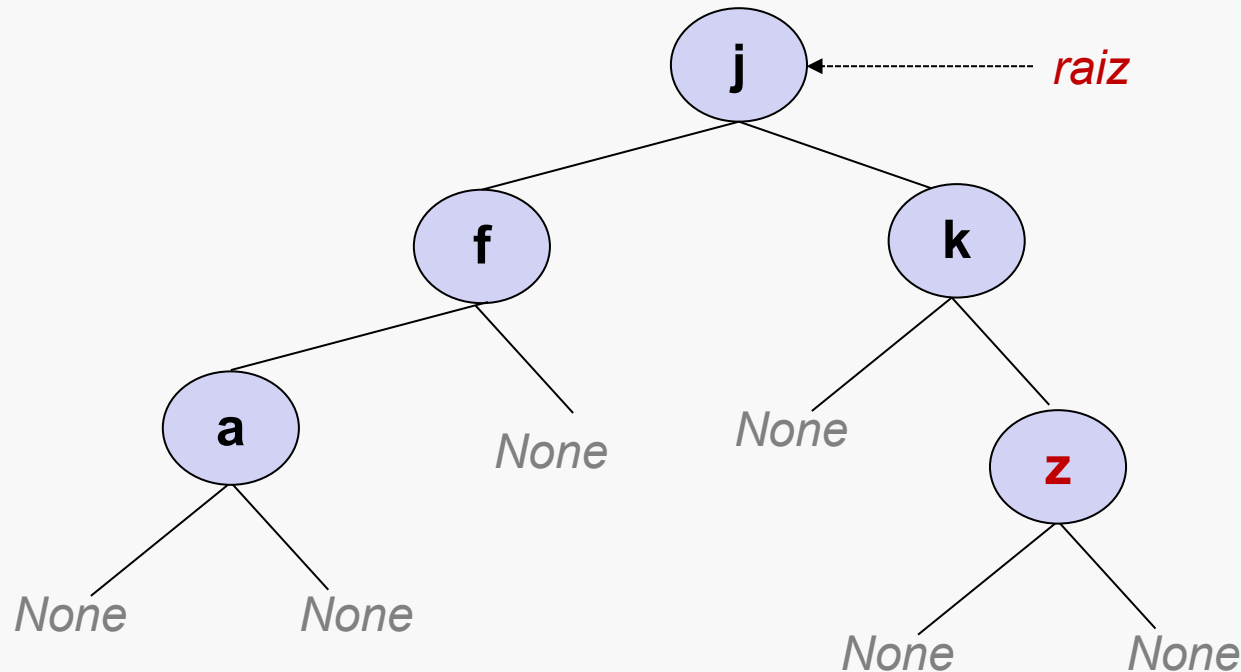
Operações – Inserir nó (5/10)

- Inserção – sequência "j" – "f" – "k" – "a" – "z" – "h"
- Inserindo o nó "a"
- Como "a" < "j", precisamos acessar a subárvore à esquerda
- E sendo "a" < "f", o novo nó será inserido à esquerda de "f"



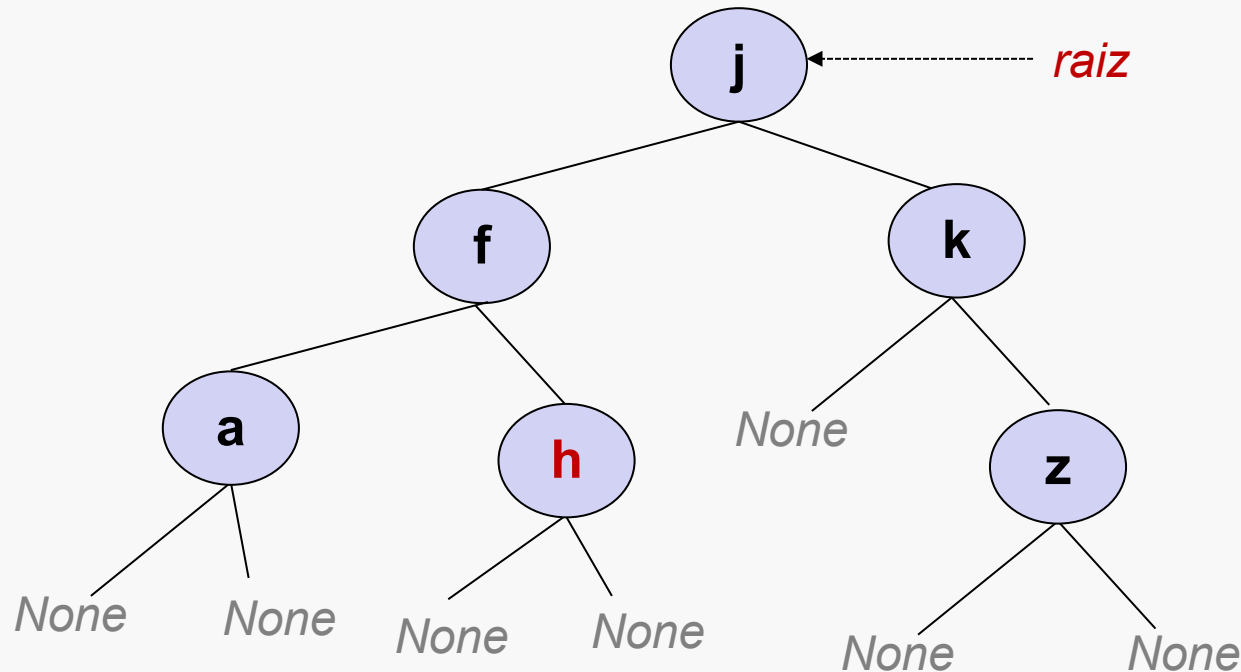
Operações – Inserir nó (6/10)

- Inserção – sequência "j" – "f" – "k" – "a" – "z" – "h"
- Inserindo o nó "z"
- Como "z" > "j", precisamos acessar a subárvore à direita
- E sendo "z" > "k", o novo nó será inserido à direita de "k"



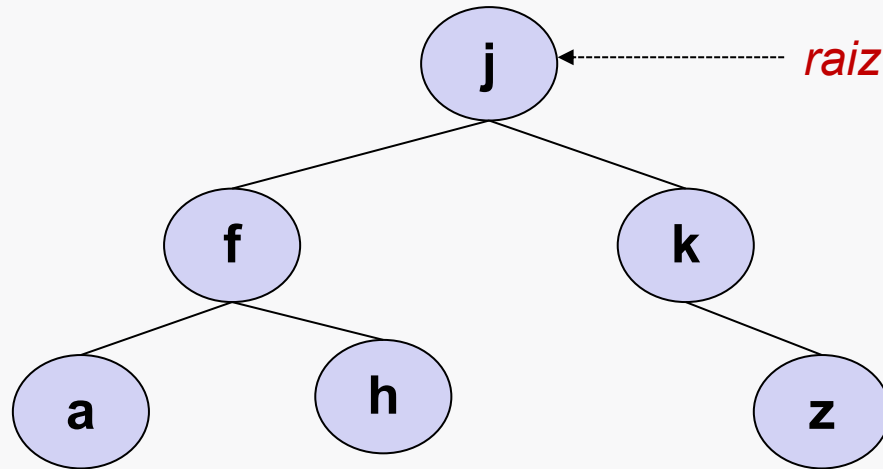
Operações – Inserir nó (7/10)

- Inserção – sequência "j" – "f" – "k" – "a" – "z" – "h"
 - Inserindo o nó "h"
 - Como "h" < "j", precisamos acessar a subárvore à esquerda
 - E como "h" > "f", o novo nó será inserido à esquerda de "f"



Introdução – Inserir nó (8/10)

- Inserção – sequência “j” – “f” – “k” – “a” – “z” – “h”
- Para a figura não ficar poluída, podemos omitir os ponteiros para *None* dos nós folha.



Operações – Inserir nó (9/10)

- Inserir (parte 1)

```
def inserir(self, x):
```

```
    # se árvore está vazia, o novo nó será a raiz
```

```
    if self.raiz is None:
```

```
        self.raiz = No(x)
```

```
        self.N += 1
```

```
        return True
```

```
    # senão, preciso achar a posição para depois inserir
```

```
    else:
```

```
        return self.inserir_no(self.raiz, x)
```

***** IMPORTANTE:** *diferente do código das outras aulas (lista, pilha, fila), nos exemplos da ABB o parâmetro x representa um dado (informação) e não um nó.*

Operações – Inserir nó (10/10)

- Inserir (parte 2)

```
def inserir_no(self, no_atual, x):
    if no_atual.info == x: return False # elemento já existe – não vou inserir

    # elemento c/ valor menor que o do nó corrente
    elif x < no_atual.info:
        if no_atual.esq is None: # se esquerda é folha, posso inserir
            no_atual.esq = No(x)
            self.N += 1
            return True
        else: # caso não seja folha, tenho que acessar subárvore à esquerda
            return self.inserir_no(no_atual.esq, x)

    # elemento c/ valor maior que o do nó corrente
    elif x > no_atual.info:
        if no_atual.dir is None: # se direita é folha, posso inserir
            no_atual.dir = No(x)
            self.N += 1
            return True
        else: # se não é folha, tenho que acessar subárvore à direita
            return self.inserir_no(no_atual.dir, x)
```

Operações – Busca

- **Buscar elemento**

- Partindo da raiz, vou procurando recursivamente nas subárvores à esquerda ou à direita, de acordo com o valor a ser buscado.

verifica se o valor x está armazenado na árvore

def buscar(self, x):

return self.buscar_no(self.raiz, x)

def buscar_no(self, no_atual, x):

se cheguei em um nó folha e não encontrei x, ele não existe na ABB

if no_atual **is** None: **return** False

elif x == no_atual.info: **return** True *# achei 😊*

elif x < no_atual.info: *# procura na subárvore à esquerda*

return self.buscar_no(no_atual.esq, x)

else: *# procura na subárvore à direita*

return self.buscar_no(no_atual.dir, x)

Operações – Caminhamento (1/9)

- Caminhamento

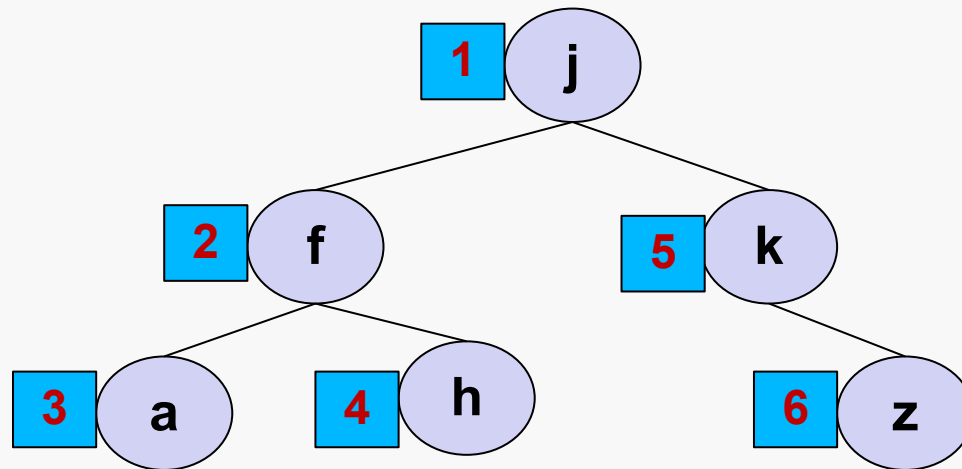
- Consiste em **visitar todos os nós** da árvore, de modo que cada nó seja visitado exatamente uma vez.
 - **Visitar** um nó = acessá-lo para realizar alguma operação.
 - Em nossos caso, o único processamento será imprimir seus dados.

```
def visitar(self, no):  
    print(no.info)
```

- Há diferentes formas de caminhar:
 - Pré-ordem (ou profundidade)
 - Pós-ordem
 - Em ordem
 - Largura

Operações – Caminhamento (2/9)

- Pré-ordem (profundidade)
 - Visita a raiz
 - Percorre a subárvore esquerda
 - Percorre a subárvore direita



"j" – "f" – "a" – "h" – "k" – "z"

Operações – Caminhamento (3/9)

- Pré-ordem (profundidade)

```
def caminhoamento_pre_ordem(self):
    if self.raiz is not None: self.pre_ordem(self.raiz)
```

```
def pre_ordem(self, no_atual):

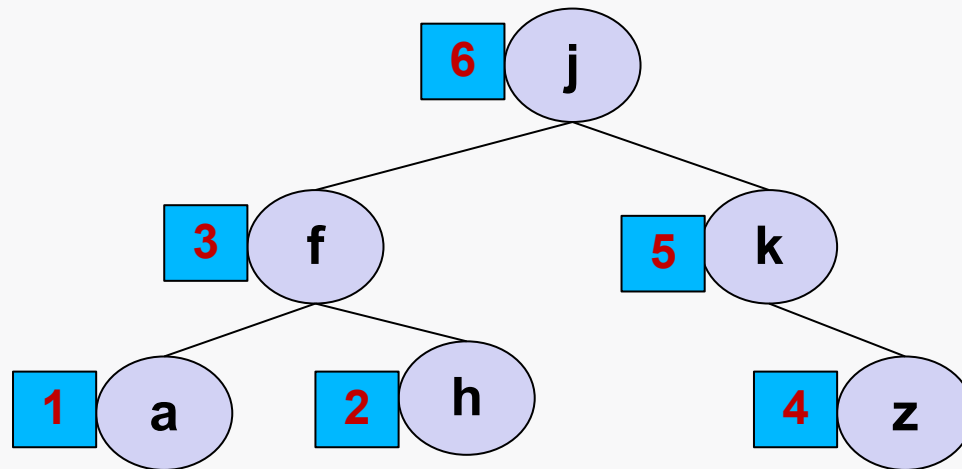
    self.visitar(no_atual)

    if no_atual.esq is not None:
        self.pre_ordem(no_atual.esq)

    if no_atual.dir is not None:
        self.pre_ordem(no_atual.dir)
```

Operações – Caminhamento (4/9)

- Pós-ordem
 - Percorre a subárvore esquerda
 - Percorre a subárvore direita
 - Visita a raiz.



"a" – "h" – "f" – "z" – "k" – "j"

Operações – Caminhamento (5/9)

- Pós-ordem

```
def caminhoamento_pos_ordem(self):
    if self.raiz is not None:
        self.pos_ordem(self.raiz)
```

```
def pos_ordem(self, no_atual):

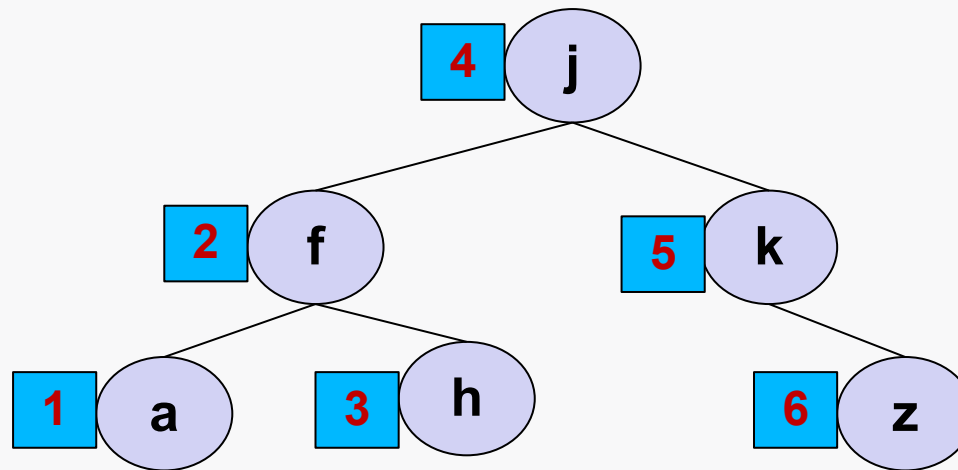
    if no_atual.esq is not None:
        self.pos_ordem(no_atual.esq)

    if no_atual.dir is not None:
        self.pos_ordem(no_atual.dir)

    self.visitar(no_atual)
```


Operações – Caminhamento (6/9)

- Em ordem
 - Caminhamento mais importante para a ABB, já que ela é uma estrutura utilizada para manter dados ordenados.
 - Percorre a subárvore esquerda
 - Visita a raiz.
 - Percorre a subárvore direita



"a" – "f" – "h" – "j" – "k" – "z"

Operações – Caminhamento (7/9)

- Em ordem

```
def caminhoamento_em_ordem(self):
    if self.raiz is not None:
        self.em_ordem(self.raiz)
```

```
def em_ordem(self, no_atual):

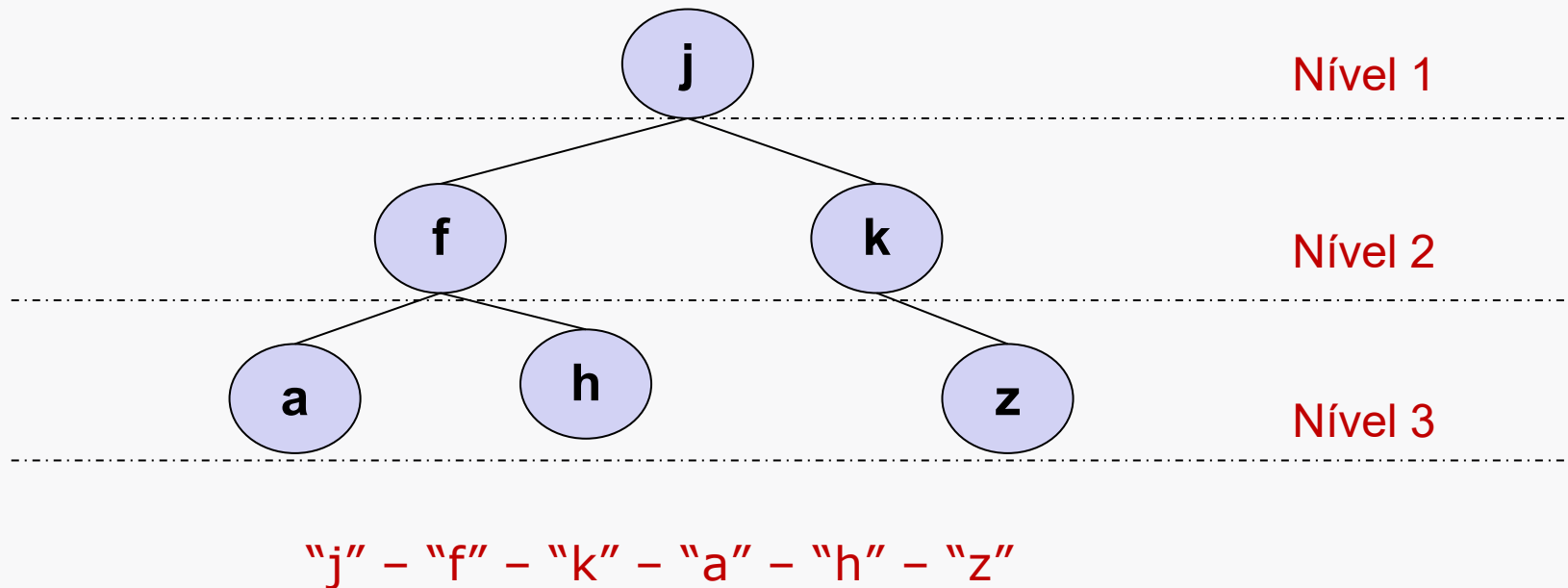
    if no_atual.esq is not None:
        self.em_ordem(no_atual.esq)

    self.visitar(no_atual)

    if no_atual.dir is not None:
        self.em_ordem(no_atual.dir)
```

Operações – Caminhamento (8/9)

- Em largura
 - Visita é feita **por nível**, da esquerda para a direita.



- A implementação requer o uso de uma ED auxiliar (ex: fila, lista)
 - Em nosso exemplo, a partir do caminhamento em pré-ordem, utilizaremos uma lista de listas para registrar cada nó e seu respectivo nível.

Operações – Caminhamento (9/9)

- Em largura

```
def caminhamento_em_largura(self):
    lst = []
    if self.raiz is not None: lst = self.em_largura(self.raiz, lst, 1)
    return lst
```

```
def em_largura(self, no_atual, lst, nivel):
    #insere o nível, caso não exista
    if nivel > len(lst): lst.append([])

    #insere o valor do nó atual
    lst[nivel-1].append(no_atual.info)

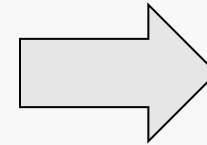
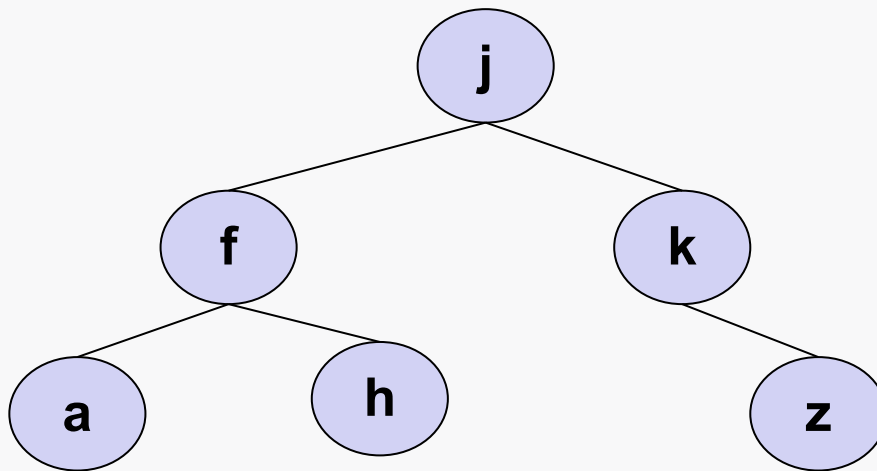
    if no_atual.esq is not None:
        self.em_largura(no_atual.esq, lst, nivel+1)

    if no_atual.dir is not None:
        self.em_largura(no_atual.dir, lst, nivel+1)

    if no_atual is self.raiz: return lst
```

Diagrama de Barras (1/2)

- Para terminar, vamos gerar uma representação string de nossa árvore através de um **diagrama de barras**.
- Como já foi feito outras vezes, implementaremos código para o método mágico **`__str__`**



j
...f
.....a
.....h
...k
.....z

Diagrama de Barras (2/2)

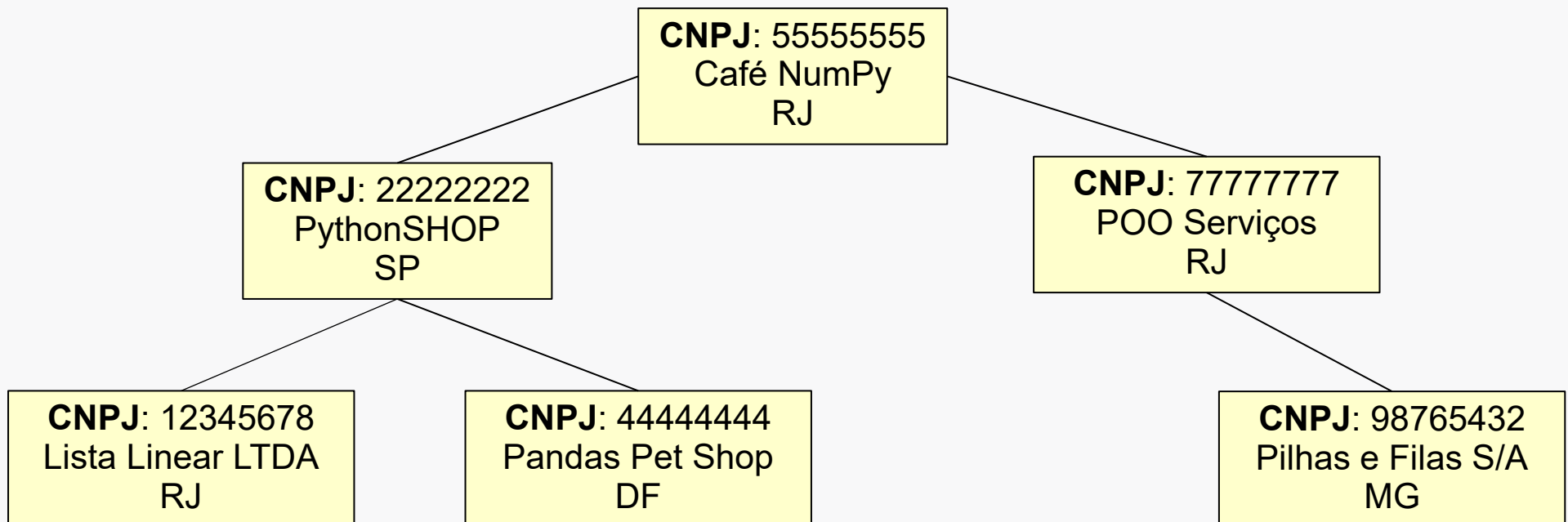
- Gerando um diagrama de barras

```
def __str__(self):  
    lst = []  
    if self.raiz is not None:  
        lst = self.diagrama_barras(self.raiz, lst, 1)  
  
    return "\n".join(lst)  
  
def diagrama_barras(self, no_atual, lst, nivel):  
  
    lst.append("..." * (nivel-1) + str(no_atual.info))  
    if no_atual.esq is not None:  
        self.diagrama_barras(no_atual.esq, lst, nivel+1)  
    if no_atual.dir is not None:  
        self.diagrama_barras(no_atual.dir, lst, nivel+1)  
  
    if no_atual is self.raiz: return lst
```

Chave de uma Árvore (1/3)

- Chave de Busca

- Muitas aplicações precisam **buscar** por um determinado **registro** em um conjunto de dados através de uma **chave**.
 - Por exemplo, buscar os dados de uma empresa que possui um determinado CNPJ
 - Neste caso, o CNPJ é a chave.

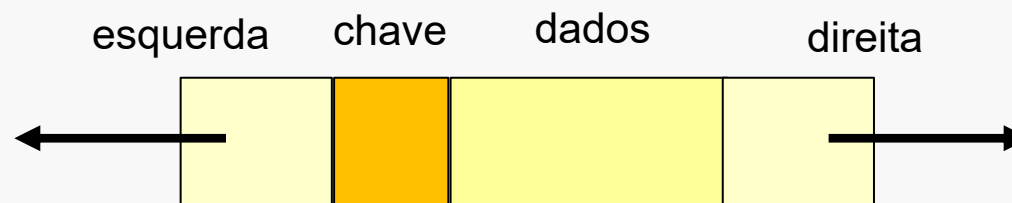


Chave de uma Árvore (2/3)

- Desta forma, podemos atualizar a definição formal de ABB:
 - Uma árvore binária **T** é uma **árvore binária de busca** (ABB) se:
 - As chaves da subárvore esquerda de T são menores do que chave da raiz de T; e
 - As chaves da subárvore da direita de T são maiores do que a chave da raiz de T; e
 - As subárvores da esquerda e da direita de T são árvores binárias de busca

Chave de uma Árvore (3/3)

- Chave de Busca
 - Para implementar, basta modificar a **estrutura do nó**:
 - **chave** (**key**): campo que define a ordem. Deve ser um identificador único de cada objeto representado no nó.
 - Ex.: CNPJ para empresa, ID ou CPF para empregado, etc.
 - **dados** (**data**): contém os demais dados o objeto representado no nó.
 - Ex.: razão social, endereço, número de empregados, etc.
 - **esquerda** (**left**): contém a referência para o nó da esquerda.
 - **direita** (**right**): contém a referência para o nó da direita.



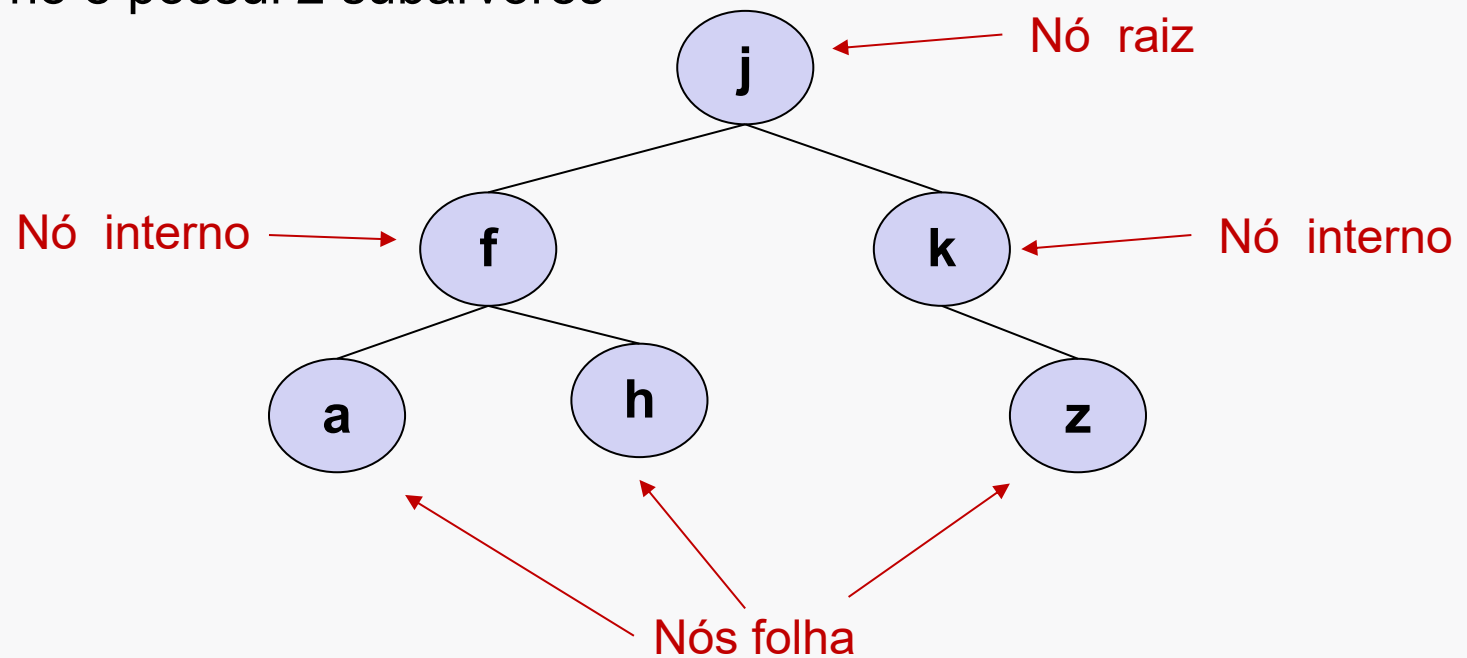
Operações – Remover Nó (1/10)

- Removendo um nó da ABB

- A remoção de um nó é bem mais complicada que a inserção ou busca.
- Vamos apresentar apenas os algoritmos.

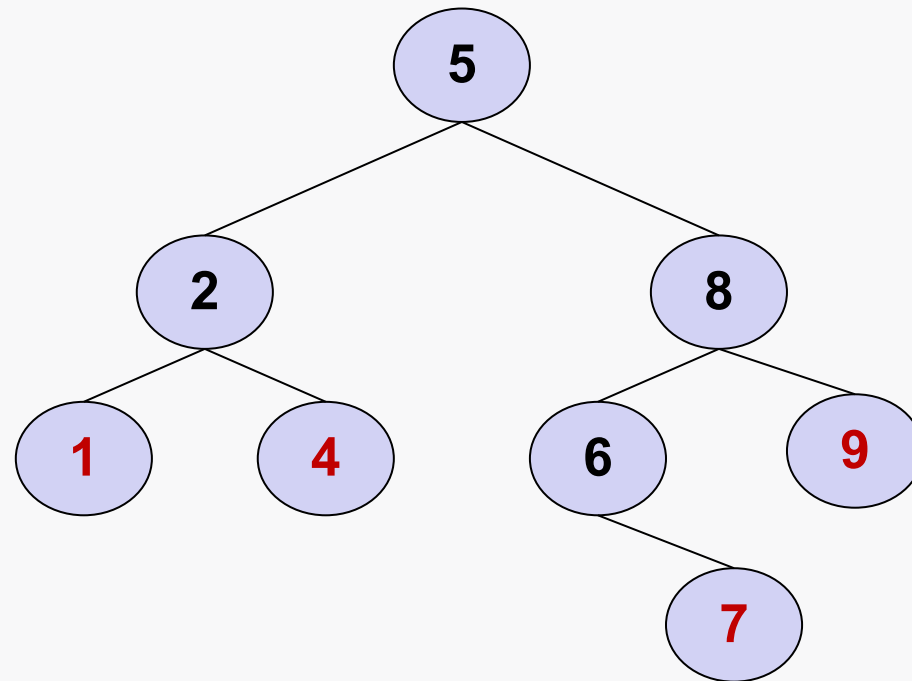
- Há **3 casos**:

- Nó é folha
- Nó é interno e possui 1 subárvore
- Nó é interno e possui 2 subárvores



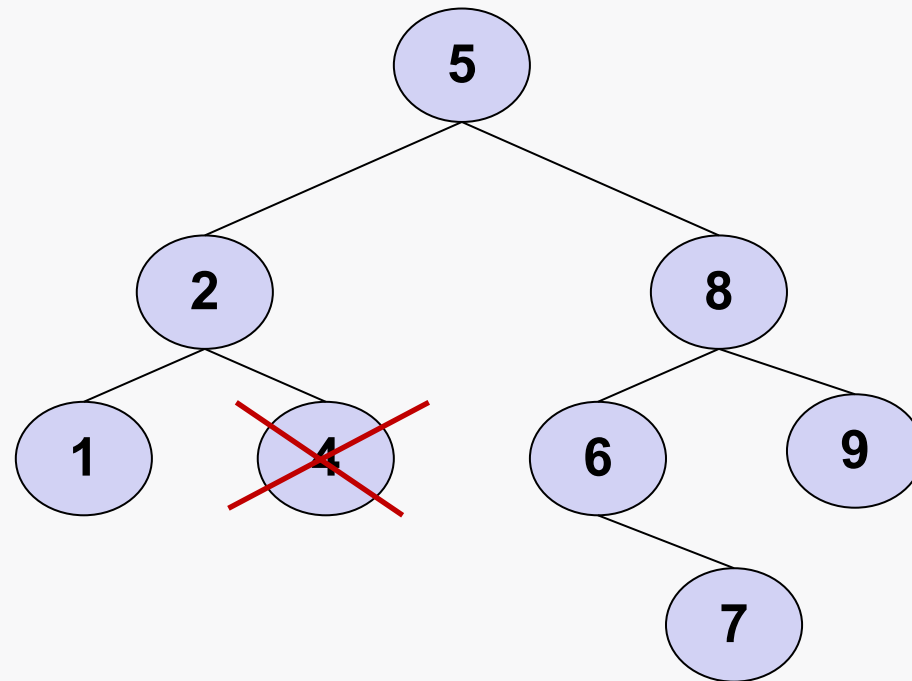
Operações – Remove Nó (2/10)

- Remove nó – caso 1: nó é folha
 - É o caso dos nós 1, 4, 7 e 9



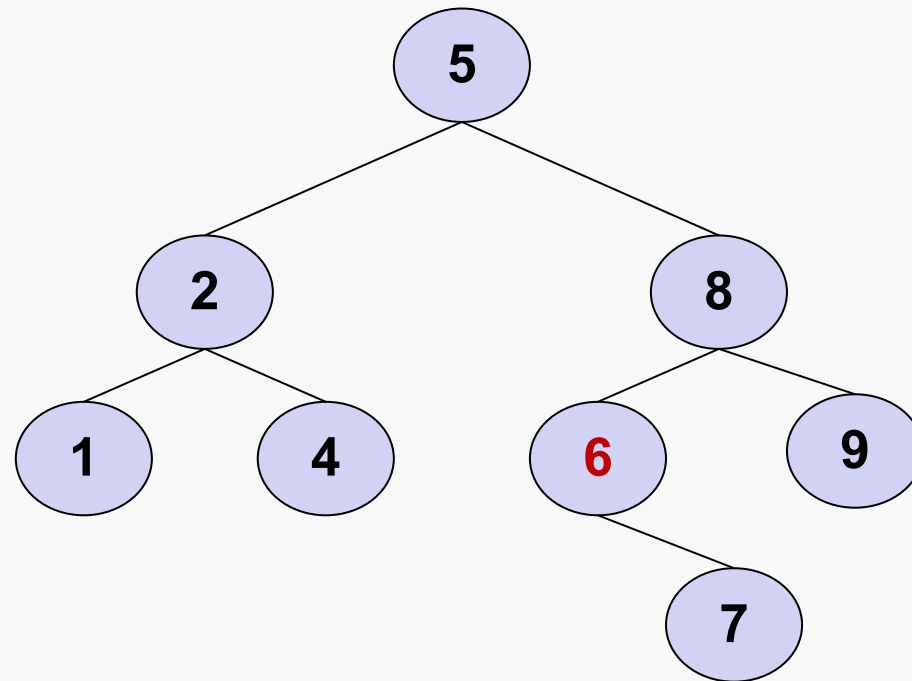
Operações – Remove Nó (3/10)

- Remove nó – caso 1: nó é folha
 - Simples. Basta remover o nó



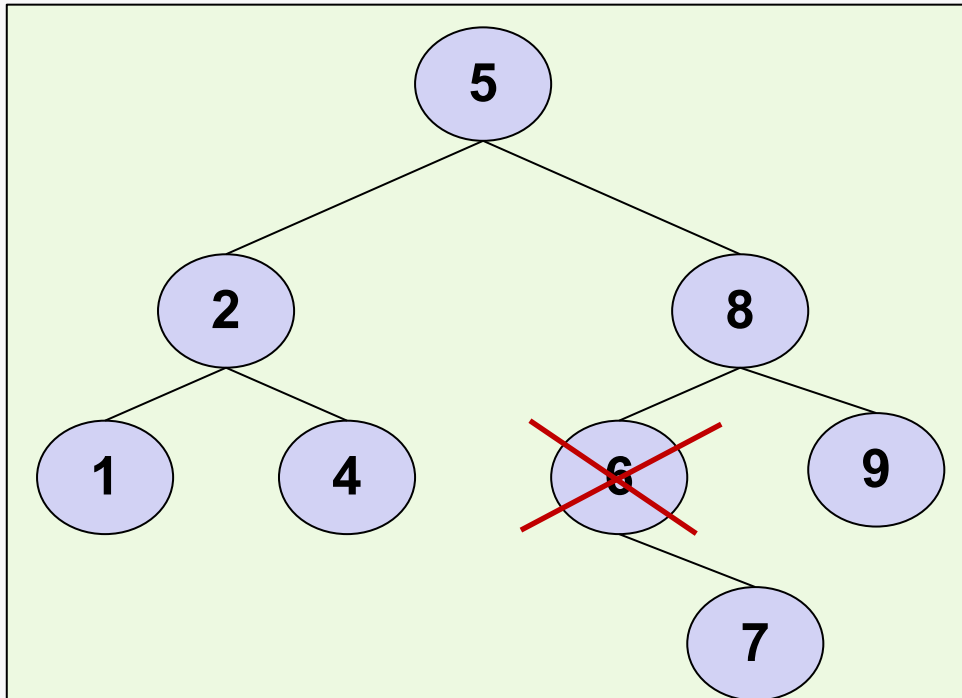
Operações – Remover Nó (4/10)

- Remover nó – caso 2: nó interno com apenas uma subárvore
 - É a situação do nó 6

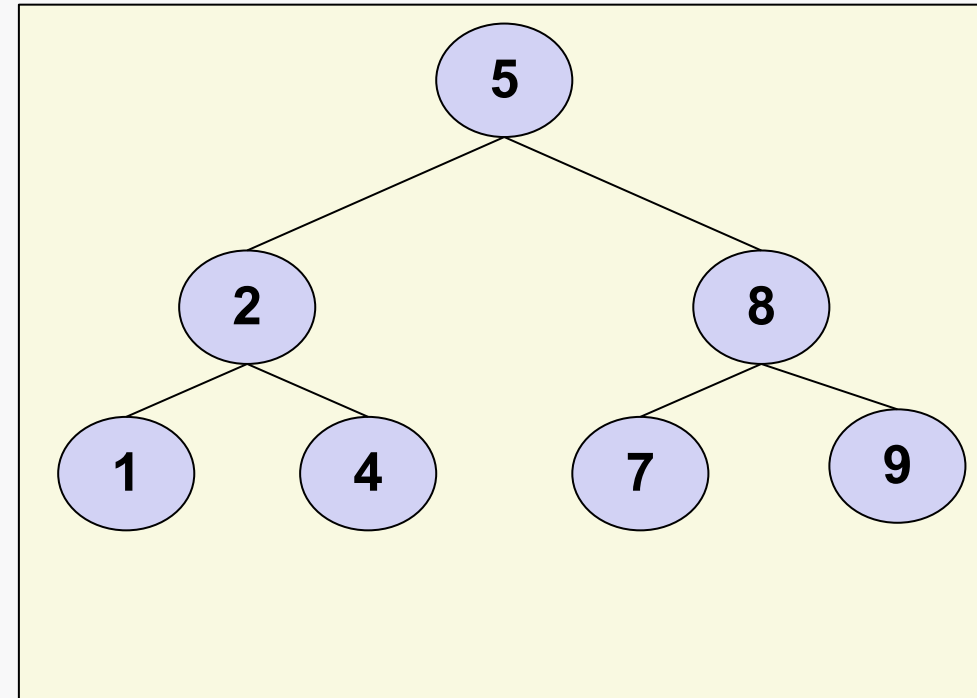


Operações – Remove Nó (5/10)

- **Remover nó – caso 2: nó interno com apenas uma subárvore**
 - Neste caso, a raiz da subárvore do nó excluído passará a ocupar o lugar do nó excluído.



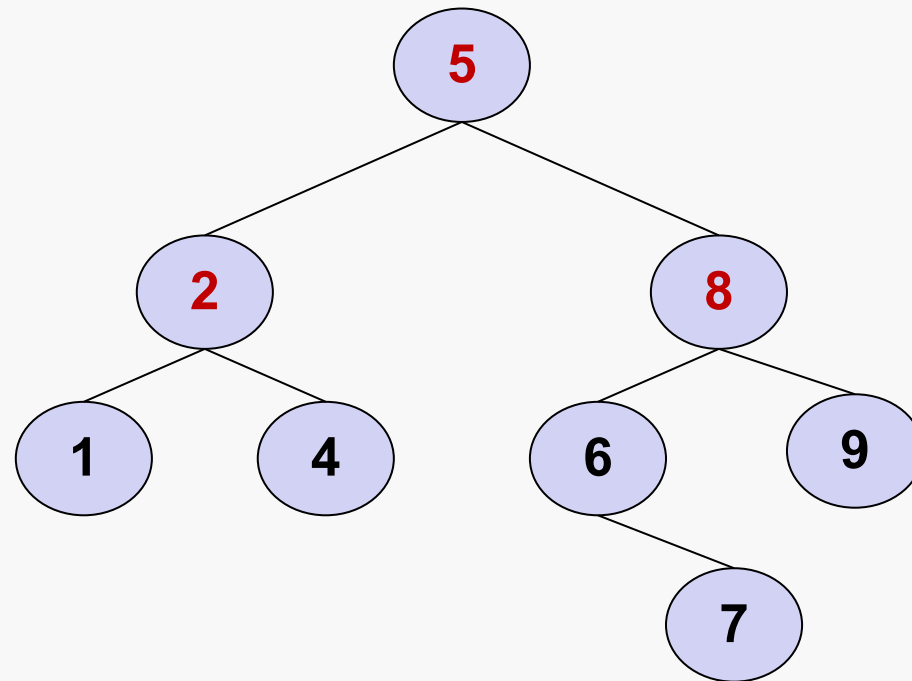
ANTES



DEPOIS

Operações – Remove Nó (6/10)

- Remove nó – caso 3: nó interno com duas subárvores
 - É o caso dos nós 2, 5 e 8



Operações – Remover Nó (7/10)

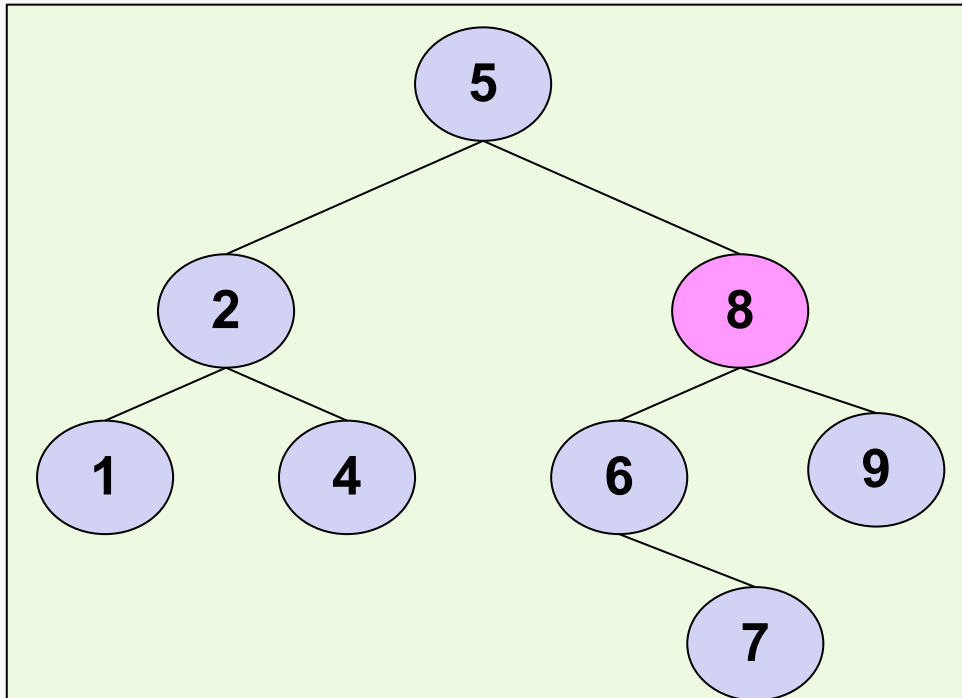
- **Remover nó – caso 3: nó interno com duas subárvores**
 - Nesta situação, será preciso reestruturar a árvore.
 - Algoritmo*:
 1. **Trocar o valor** do nó a ser removido com:
 - o valor do nó que tenha a **maior chave** na subárvore à **esquerda** (*adotaremos esse*); OU
 - o valor do nó que tenha a **menor chave** na subárvore à direita.
 2. Ir até a subárvore onde foi feita a troca e **remover o nó**.
 - Devido à ordenação da ABB, o nó a ser removido sempre terá no máximo uma subárvore.

* Edelweiss e Galante (2009).

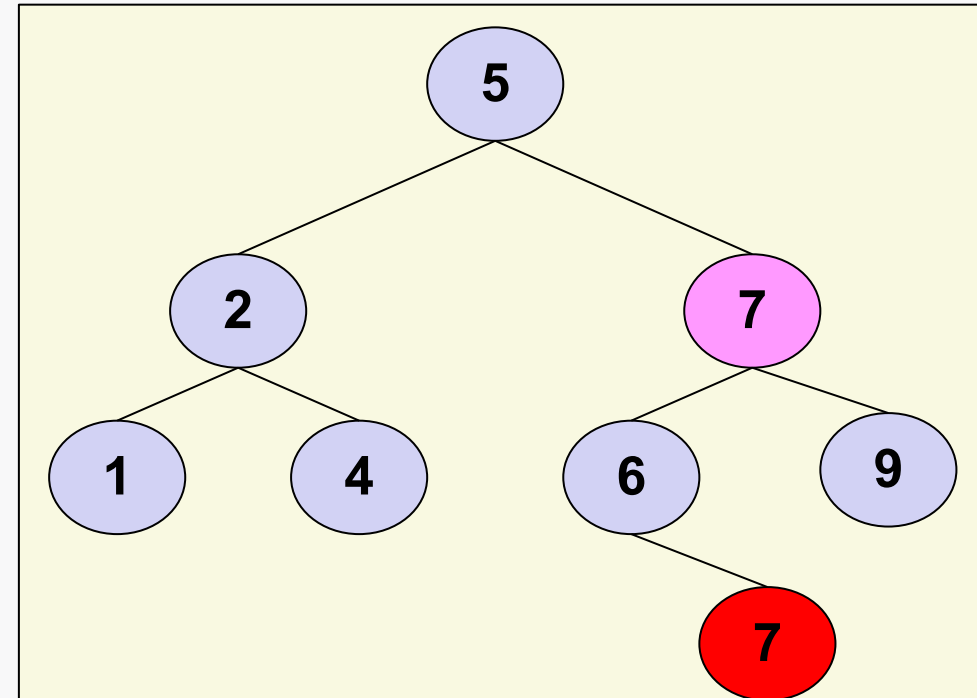
Operações – Remove Nó (8/10)

- Remove nó – caso 3: nó interno com duas subárvores
 - Exemplo remoção do nó 8

PASSO 1: Trocar o valor 8 por 7 (maior valor da sub-árvore à esquerda)



SITUAÇÃO
ORIGINAL

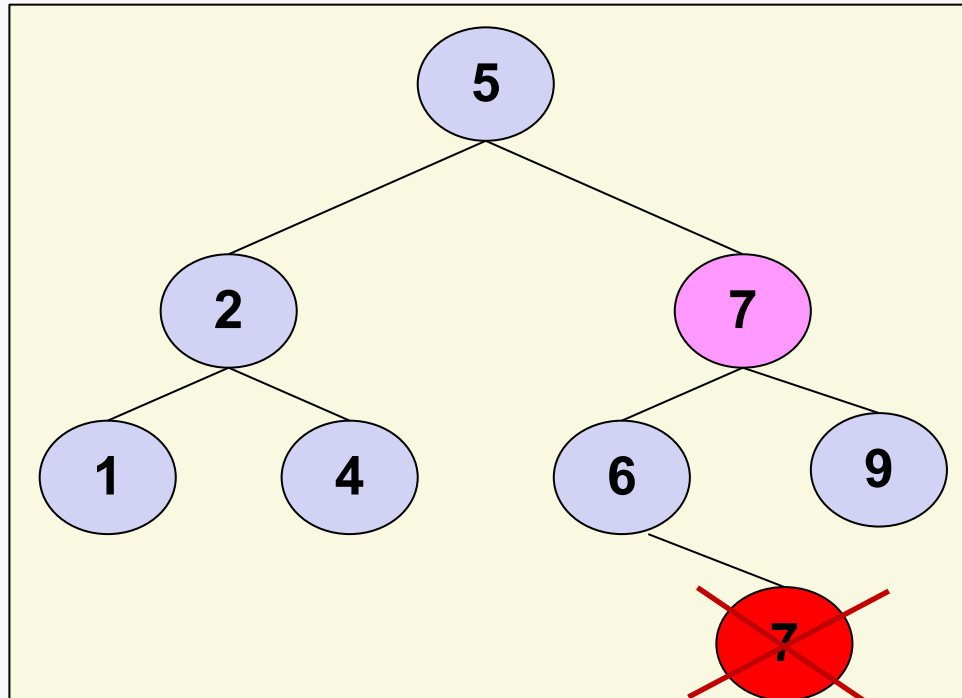


SITUAÇÃO
APÓS PASSO 1

Operações – Remove Nó (9/10)

- **Remover nó – caso 3: nó interno com duas subárvores**
 - Exemplo remoção do nó 8

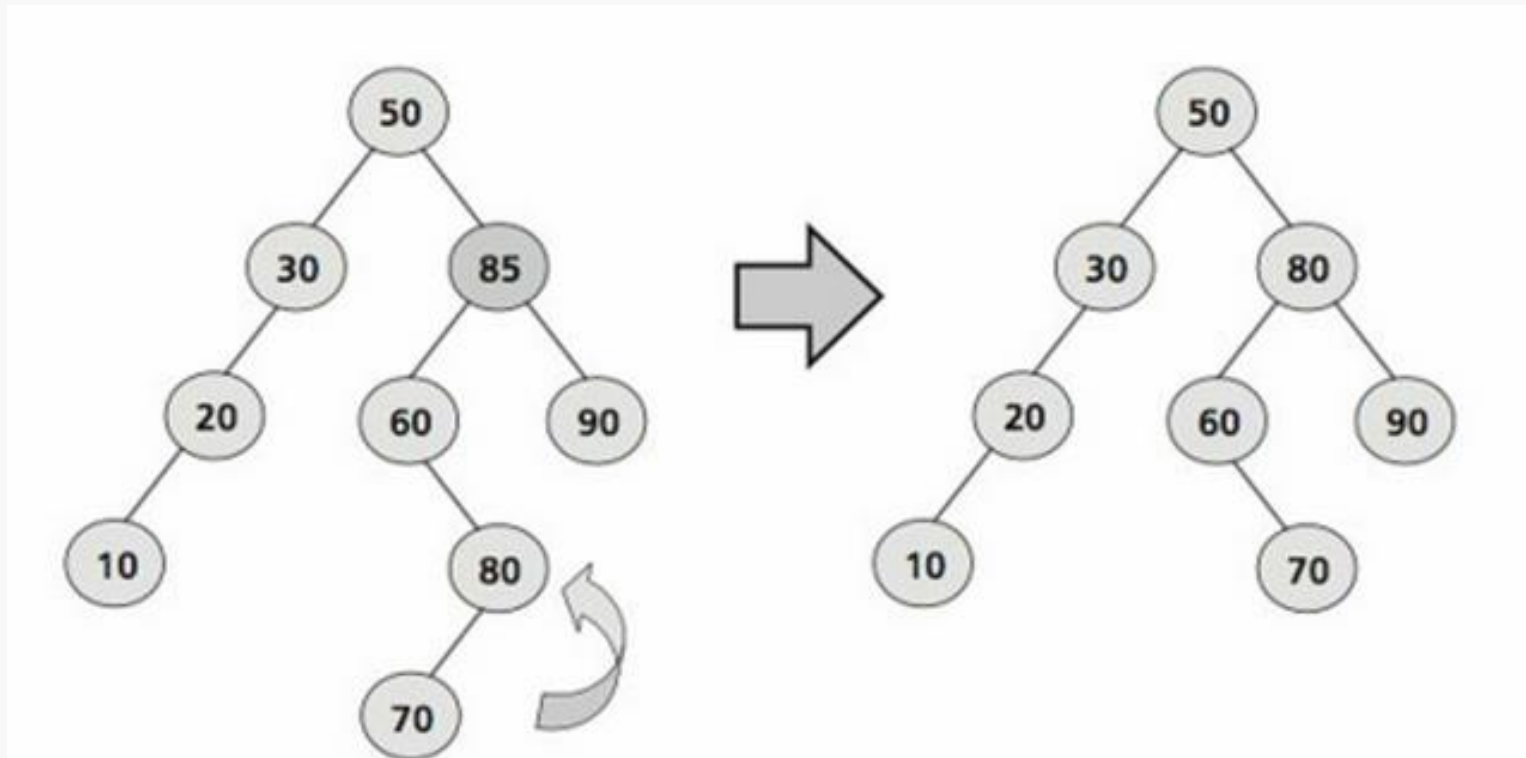
PASSO 2: ir até a subárvore onde foi feita a troca e remover o nó



SITUAÇÃO
APÓS PASSO 2
(FINAL)

Operações – Remove Nó (10/10)

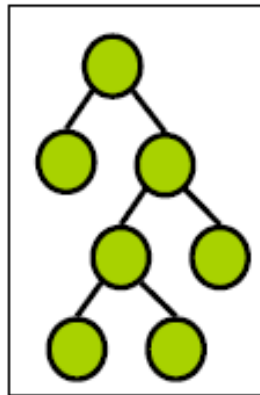
- Remove nó – caso 3: nó interno com duas subárvores
 - Outro exemplo (Edelweiss e Galante, 2009):
 - Remoção do nó 85



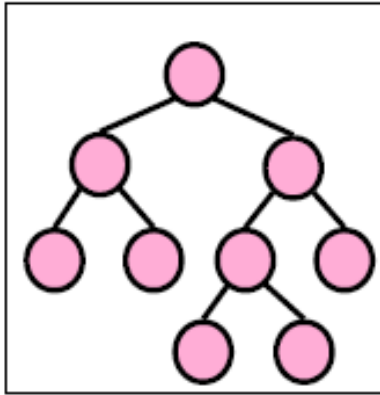
Árvores Balanceadas (1/5)

- Tipos de ABBs

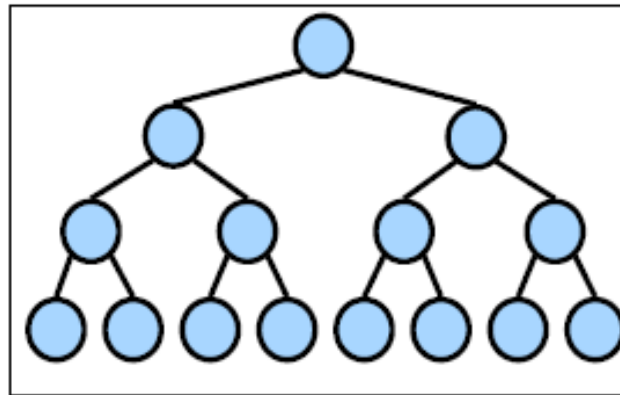
- De acordo com a ordem em que os nós são inseridos, as ABBs podem ficar com 4 diferentes formatos:



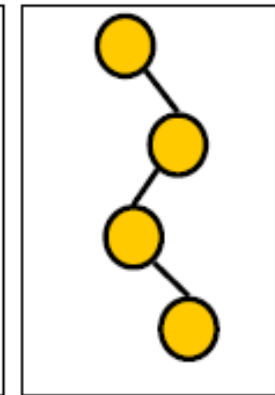
Estritamente Binária
0 ou 2 filhos



Binária Completa
Sub-árvores vazias apenas no último ou penúltimo nível



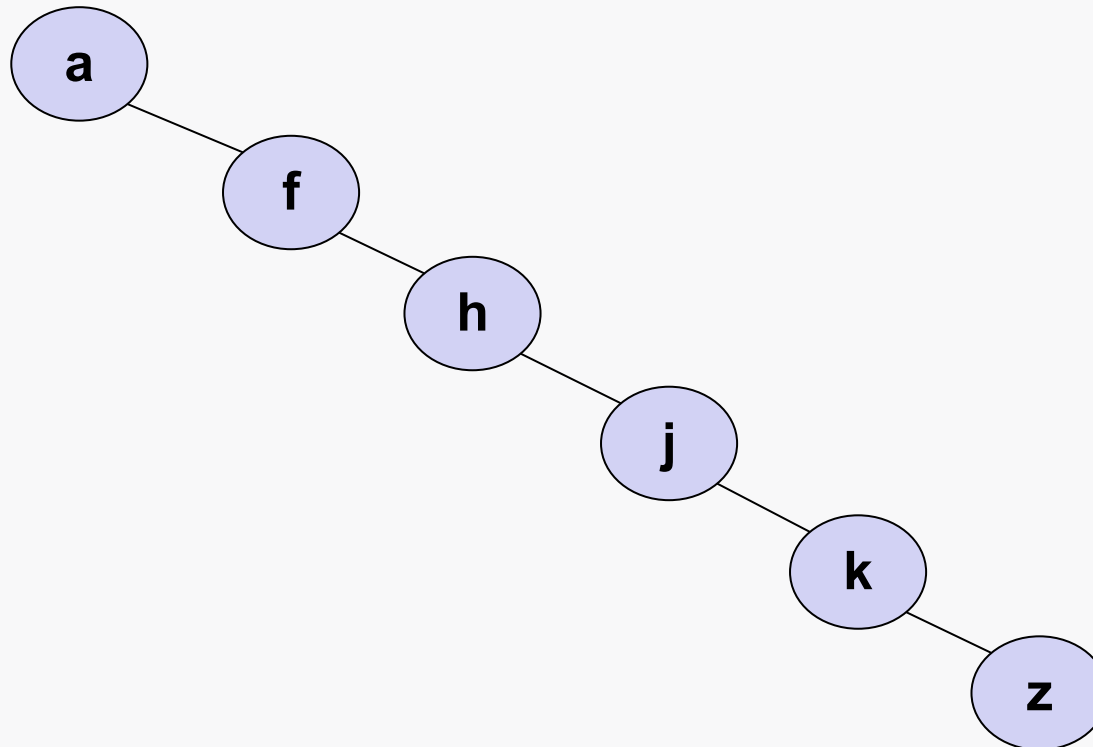
Binária Cheia
Sub-árvores vazias somente no último nível



Zigue Zague
Nós internos com 1 subárvore vazia

Árvores Balanceadas (2/5)

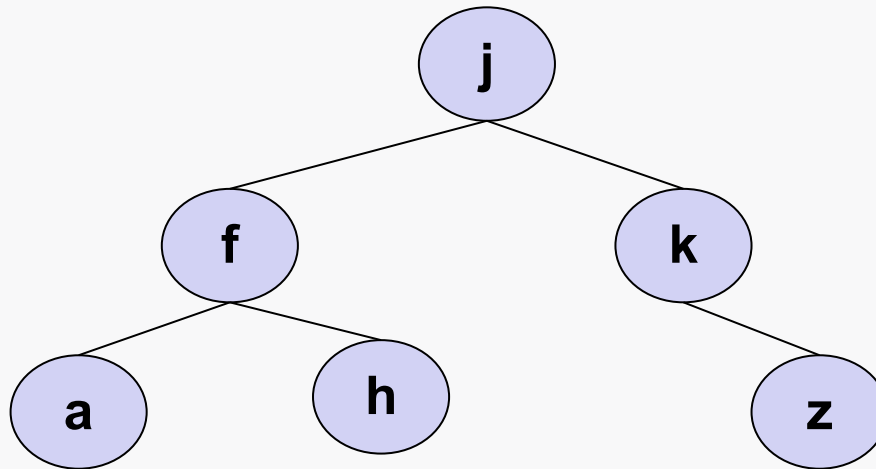
- **Problema da ABB – Desbalanceamento Progressivo**
 - Dependendo da ordem em que os nós são inseridos na ABB, a topologia pode ficar ruim, o que significa uma árvore em que as **buscas** ficam mais **custosas**.
 - Por exemplo, na árvore abaixo, a busca custa $O(n)$



Árvores Balanceadas (3/5)

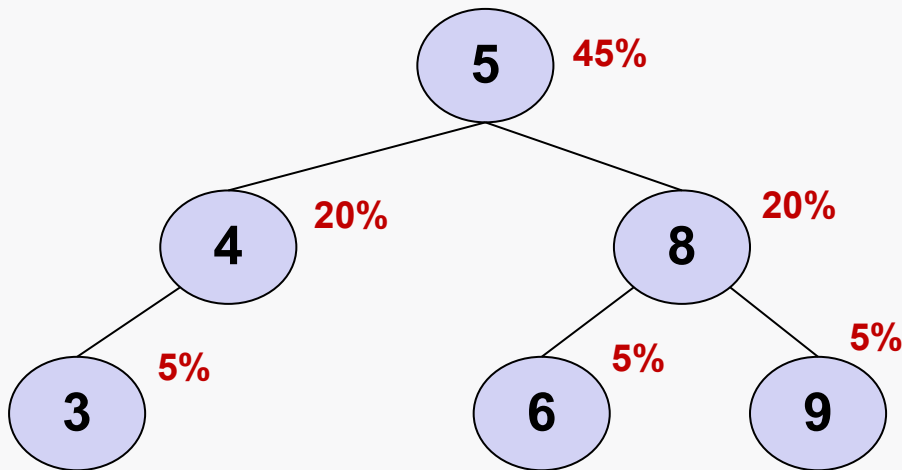
- **Distribuição Equilibrada**

- Por outro lado, em uma árvore com distribuição equilibrada dos nós, como a binária completa e a binária cheia, a busca é mais eficiente.
 - Número de comparações será $O(\log n)$.
- Por essa razão, na prática, os softwares fazem uso de **árvores balanceadas** em vez de simples ABBs.

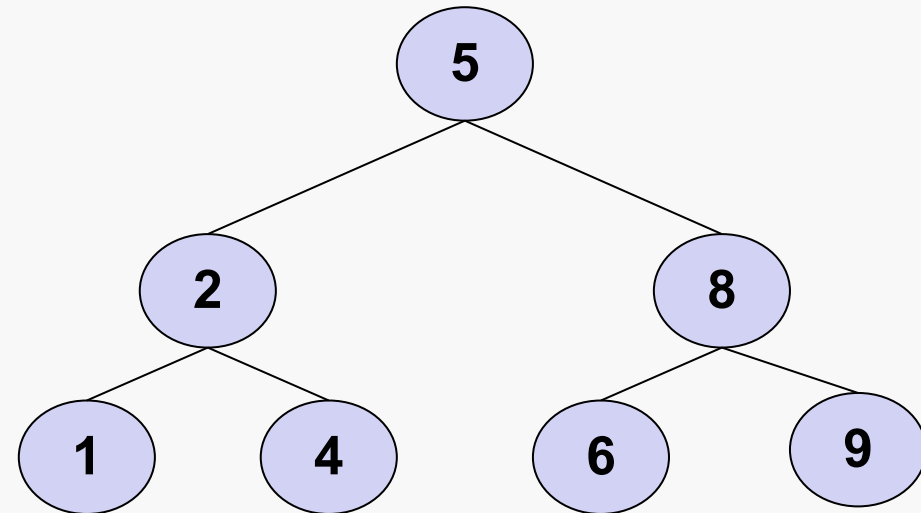


Árvores Balanceadas (4/5)

- Árvores Balanceadas - Estratégias
 - Há **duas estratégias** possíveis:



- **balanceamento por frequência:** nós mais acessados ficam dispostos mais próximos à raiz da árvore.



- **balanceamento por altura:** todas as folhas ficam à mesma distância da raiz, ou a uma distância muito semelhante.

Árvores Balanceadas (5/5)

- **Operações – Inserção / Remoção**
 - Sempre que for efetuada uma operação de inserção ou remoção de um nó em uma árvore balanceada, deve ser feita uma verificação de toda a árvore, para verificar se ela continua balanceada.
 - Caso não esteja, a árvore deve ser **reestruturada** para que volte a ser balanceada
 - Exemplos de árvores balanceadas:
 - Árvore AVL – balanceamento por altura
 - Splay – balanceamento por frequência

Resumo

- Estruturas de Dados – Caracterização na linguagem Python

ED	Tipo Básico	ED Nativa	Sequência	Iterável	Mutável
Árvore				X	X
bool, int, float	X	X			
Conjunto (set)		X		X	X
DataFrame (pandas)			X	X	X
Dicionário (dict)		X		X	X
Fila					X
Grafo				X	X
Lista (list)		X	X	X	X
Lista Encadeada			X	X	X
ndarray (NumPy)			X	X	X
Pilha					X
Series (pandas)			X	X	X
str (String)	X	X	X	X	
Tupla (tuple)		X	X	X	

Referências

- Edelweiss, N., Galante, R. (2009). “Estruturas de Dados”. Bookman (*Caps. 5 e 6*).
- Goodrich, M. T., Tamassia, R. e Goldwasser, M. H. (2013). “Data Structures and Algorithms in Python”. Wiley. (*Cap. 11*)
- Braganholo, V. (2020). Árvores Binárias de Busca
<https://sites.google.com/ic.uff.br/estrutura-de-dados-icuff/home>
- Pitts, R. I. (2000). Trees. <https://www.cs.bu.edu/teaching/c/tree/>